SISTEMAS DISTRIBUÍDOS · UFRRJ · 2026

Biblioteca RPC

Um sistema de empréstimo de livros em que cliente e servidor conversam por **chamada de procedimento remoto**, com gRPC.

Natã Lopes Martins
Prof. Ubiratam de Paula

gRPC TypeScript SQLite TLS

O PROBLEMA

Tema 5 do enunciado: empréstimo de livros em biblioteca

AS CINCO FUNÇÕES PEDIDAS

```
realizar_emprestimo(usuario, livro, data)
devolver_livro(usuario, livro)
consultar_livro(livro)
consultar_emprestimos_usuario(usuario)
consultar_emprestimos_livro(livro)
```

Cada uma com as mensagens de erro do enunciado: “Livro indisponível”, “Usuário com limite de empréstimos atingido”, “Livro já devolvido”...

O QUE O TRABALHO EXIGE

Cliente/servidor com RPC

O cliente chama funções que executam no servidor.

Servidor guarda os dados

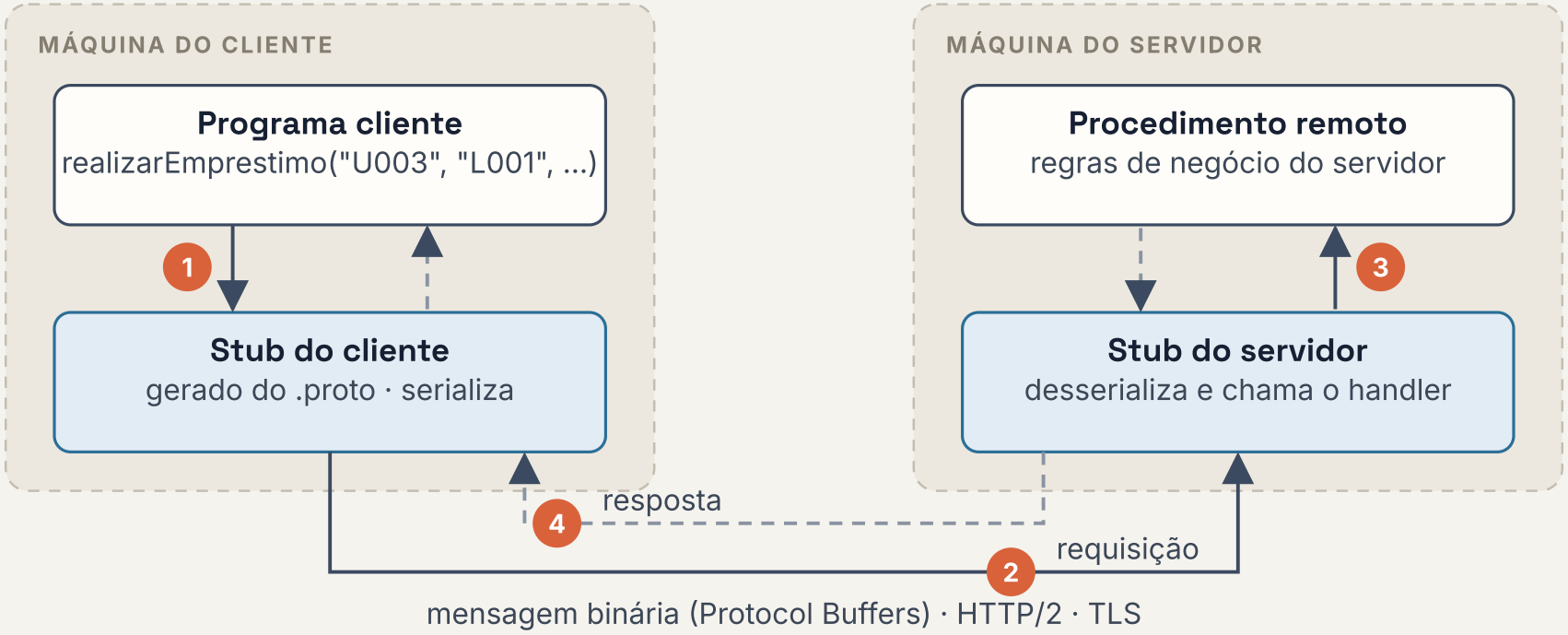
De forma simples, e expõe a API por RPC.

Cliente executa cada função

Interface de texto ou gráfica.

Uma chamada na Biblioteca RPC

O caminho de um pedido de empréstimo, do cliente ao servidor e de volta.



1 O stub do cliente **serializa** os argumentos

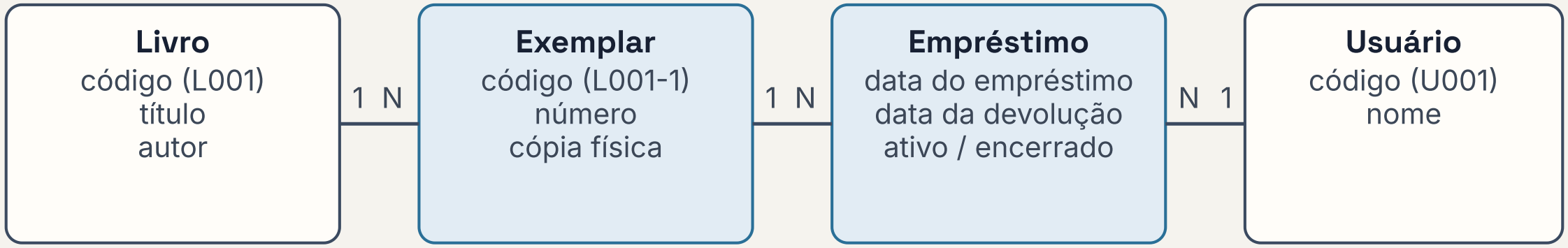
2 A mensagem atravessa a **rede**

3 O stub do servidor **chama a função**

4 A **resposta** volta; a rede pode falhar ou demorar

MODELO DO DOMÍNIO

Livro, Exemplar, Usuário e Empréstimo



Um Exemplar tem no máximo um Empréstimo ativo; um Usuário tem no máximo 3.

7 dias
de prazo; depois disso, o empréstimo está **atrasado**

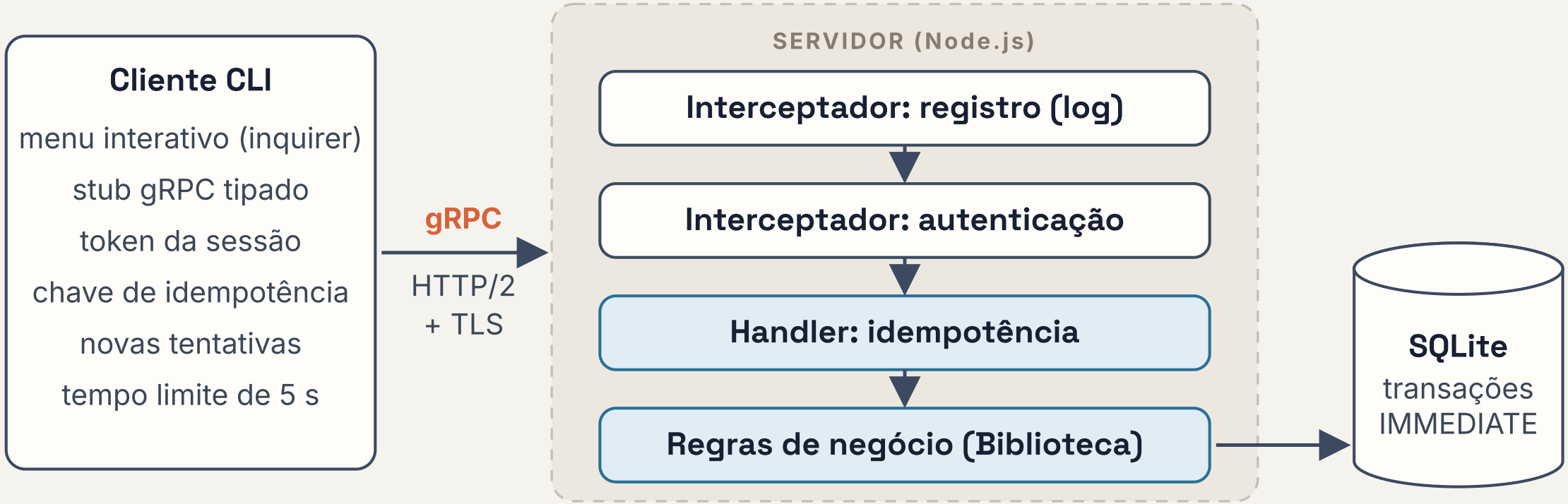
3
empréstimos ativos, no máximo, por usuário

1
exemplar do mesmo livro por usuário: a devolução nunca é ambígua

L001-2
o servidor escolhe o exemplar livre de **menor número**

ARQUITETURA

Cliente CLI, servidor gRPC e banco SQLite



Cada chamada passa por interceptadores (o *middleware* do gRPC) antes das regras de negócio. Só o servidor acessa o banco.

FERRAMENTAS E TECNOLOGIAS

O que foi usado e por quê

gRPC

Middleware de RPC usado na indústria: stubs, metadados, interceptadores, prazos e novas tentativas.

Protocol Buffers + ts-proto

O `.proto` é a IDL: contrato único do qual saem os stubs e os tipos.

TypeScript + Node.js 22

Uma linguagem nos dois lados, com tipagem estática ligada ao contrato.

SQLite

"Armazenar de forma simples": um arquivo só, mas com transações e índices.

TLS + scrypt

Canal criptografado e senha guardada como *hash*.

Inquirer · Vitest

Menus navegáveis no terminal e **63 testes** de ponta a ponta sobre um servidor real.

CONTRATO DA API

Do `.proto` ao código: um contrato, dois lados

```
// proto/biblioteca.proto
service Biblioteca {
  rpc RealizarEmprestimo(RealizarEmprestimoRequest)
      returns (RealizarEmprestimoResponse);
  rpc DevolverLivro(DevolverLivroRequest)
      returns (DevolverLivroResponse);
  // … mais 7 métodos
}

message RealizarEmprestimoRequest {
  string codigo_usuario  = 1;
  string codigo_livro    = 2;
  string data_emprestimo = 3; // AAAA-MM-DD
}
↑ 1, 2, 3 são etiquetas: na rede vai o número,
  não o nome. Um campo novo ganha o 4, e
  clientes antigos continuam funcionando.
```

`npm run proto`

→

ts-proto

BibliotecaClient

Stub do cliente: cada método vira uma função.

BibliotecaService

Definição que o servidor implementa.

Tipos TypeScript

Cliente e servidor não podem divergir sobre o formato das mensagens.

A API

As 5 funções do enunciado, mais 4 de apoio

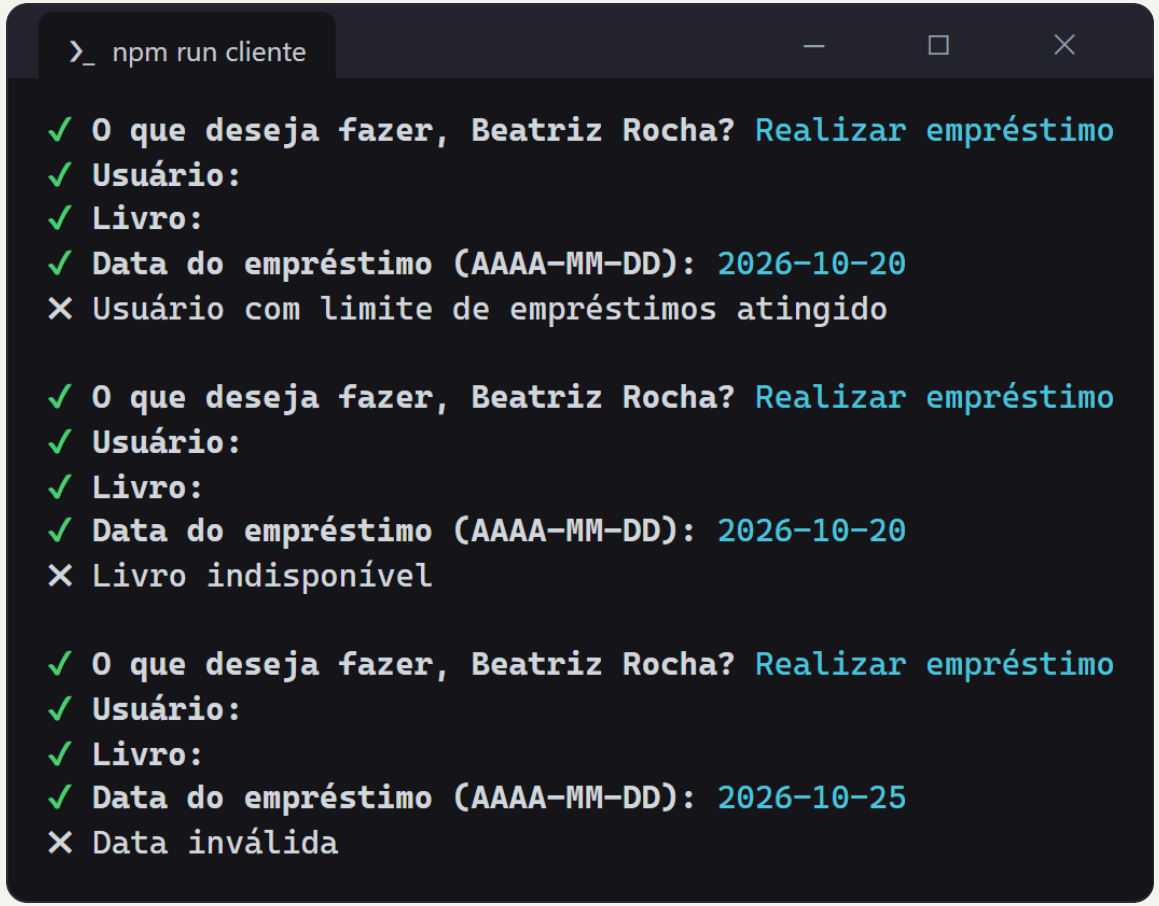
Regra: o que mostra ou altera dados de pessoas exige login; o que é do acervo é público.

MÉTODO	O QUE FAZ	ACESSO
`RealizarEmprestimo`	Empresta o exemplar livre de menor número; devolve a data limite	exige login
`DevolverLivro`	Encerra o empréstimo ativo, com a data de hoje do servidor	exige login
`ConsultarLivro`	Disponibilidade e situação de cada exemplar	público*
`ConsultarEmprestimosUsuario`	Empréstimos ativos, no prazo ou atrasados	exige login
`ConsultarEmprestimosLivro`	Histórico de todos os exemplares do livro	exige login
`Entrar`	Login do bibliotecário; devolve o token da sessão (8 h)	público
`ListarLivros`	Catálogo com a disponibilidade de cada livro	público
`ListarUsuarios`	Usuários cadastrados, para a lista de seleção do cliente	exige login
`ObterHoje`	Data do servidor, usada como padrão no empréstimo	público

* Sem login, `ConsultarLivro` não revela quem está com cada exemplar.

ERROS

Cada erro do enunciado vira um *status code* do gRPC



`NOT_FOUND`

Usuário, livro ou empréstimo inexistente

`INVALID_ARGUMENT`

Data mal formatada ou futura

`FAILED_PRECONDITION`

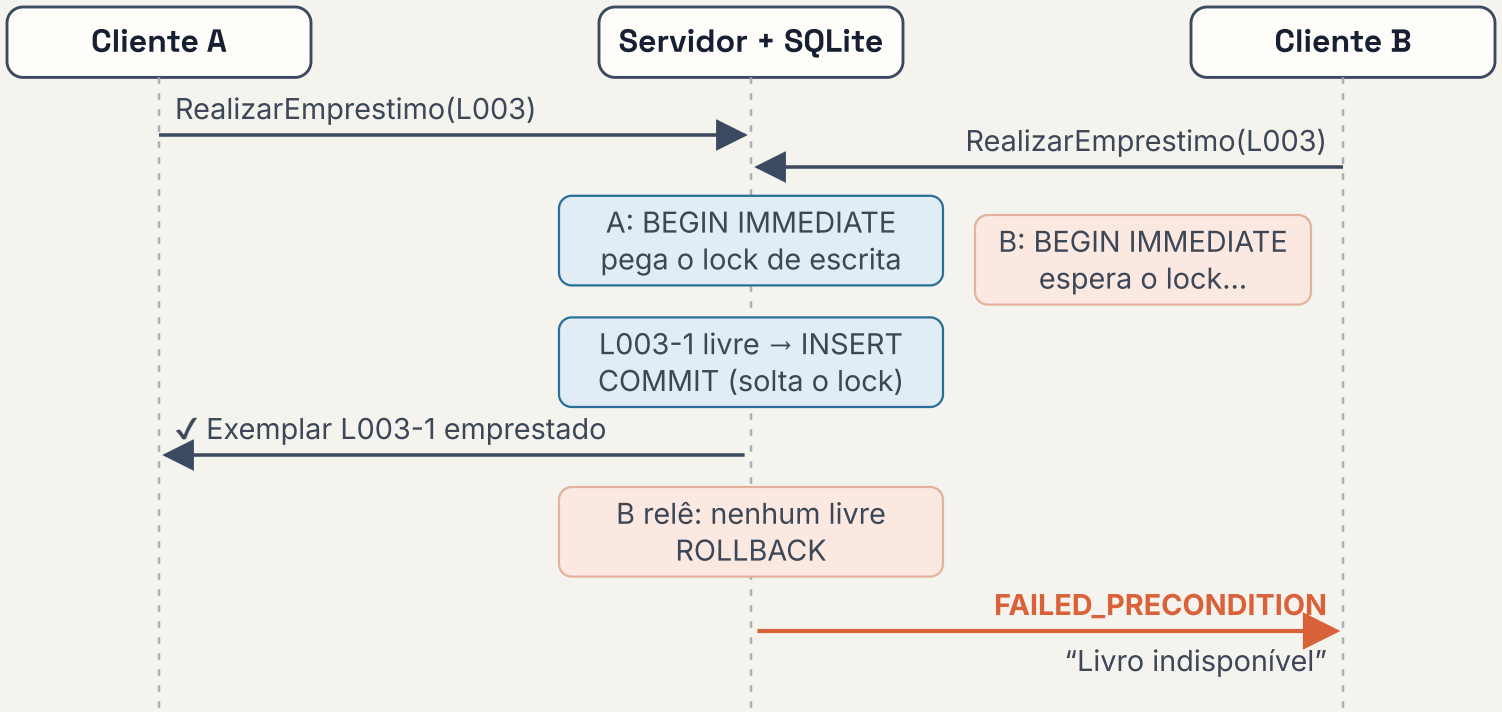
Limite atingido, livro indisponível, livro já devolvido

`UNAUTHENTICATED`

Sem login ou sessão expirada

A mensagem do enunciado vai junto, no campo `details`.
Consulta sem resultado não é erro: devolve lista vazia.

Dois clientes, um único exemplar: só um leva



Exclusão mútua no banco

`BEGIN IMMEDIATE` pega o lock de escrita **antes** de ler a disponibilidade. O segundo pedido espera e relê.

Segunda garantia

Índice único: no máximo um empréstimo ativo por exemplar.

Testado

5 pedidos simultâneos pelo último exemplar → 1 sucesso, 4 "Livro indisponível".

SEGURANÇA

Canal criptografado, login e acesso negado por padrão

TLS

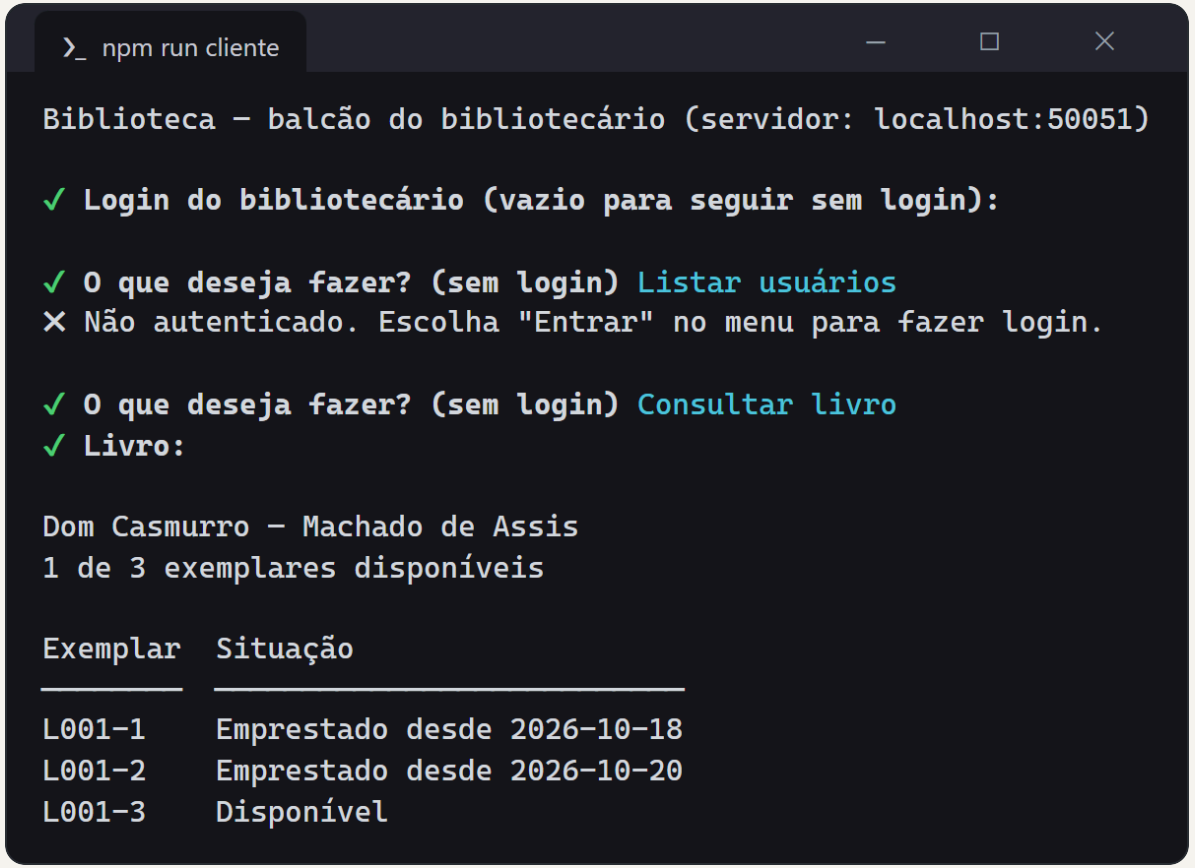
Senha e token nunca trafegam em texto puro. Certificado autoassinado, gerado localmente e fora do git.

Login do bibliotecário

Senha com *hash* scrypt e sal; token aleatório de sessão válido por 8 horas.

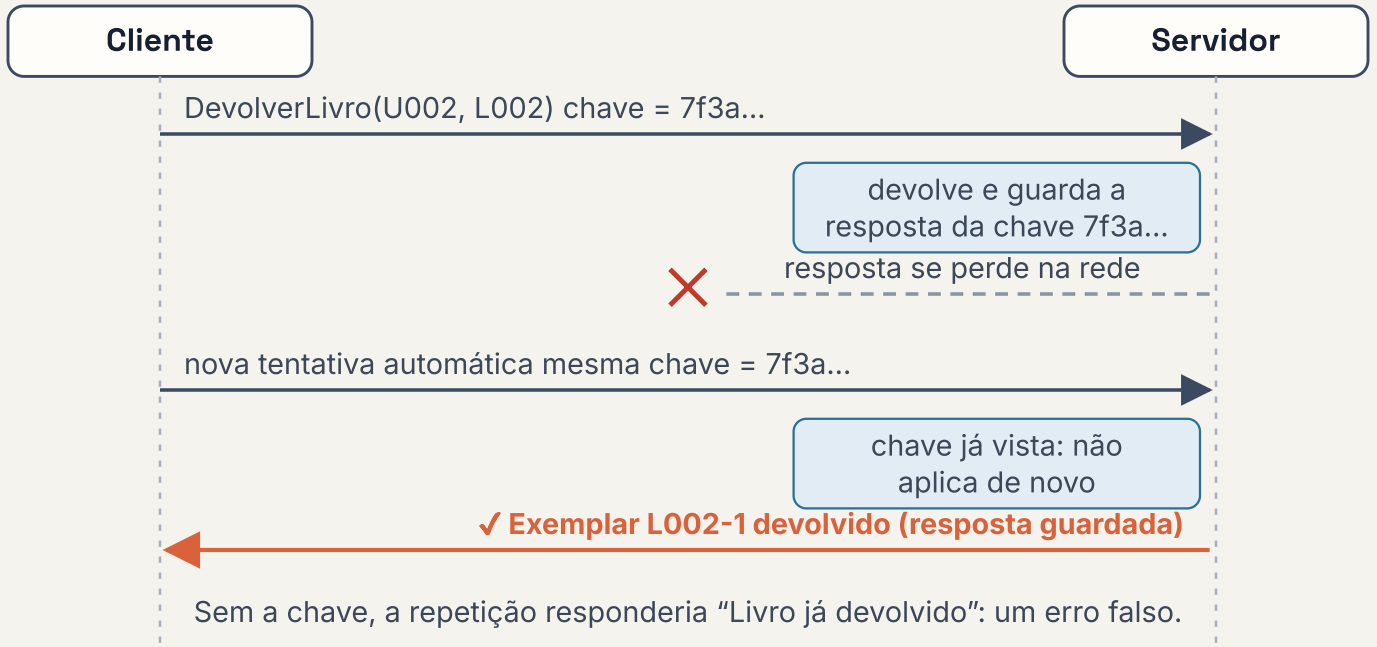
Interceptador que nega por padrão

Só uma lista de métodos é pública. Um método novo já nasce protegido.



Sem login: a função protegida é recusada pelo servidor, e a consulta pública esconde quem está com o livro.

Repetir quando a rede falha, sem aplicar duas vezes



Novas tentativas

Política nativa do gRPC: até 5 tentativas, com espera crescente, só quando o servidor está `UNAVAILABLE`.

Chave de idempotência

Um UUID por escrita. O servidor guarda a resposta na mesma transação da operação.

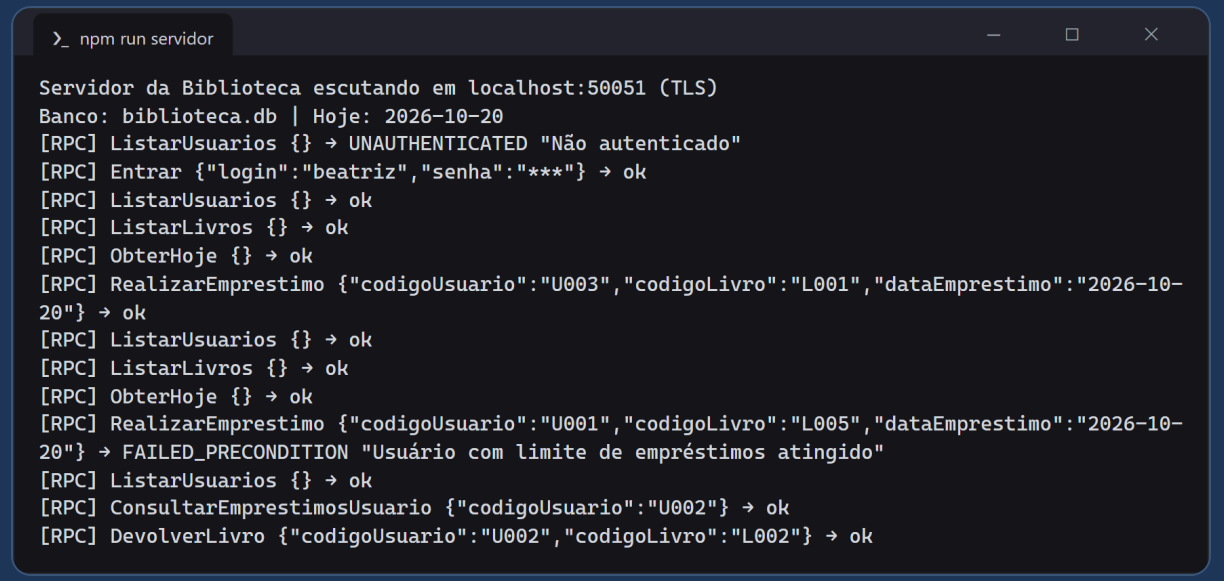
E mais

Transações atômicas, dados em disco e prazo de 5 s por chamada.

DEMONSTRAÇÃO

Ao vivo: dois terminais, um servidor

1. Subir o servidor e acompanhar o **log das chamadas**
2. Entrar como bibliotecária e **consultar um livro**
3. **Realizar um empréstimo** e ver o exemplar escolhido
4. Provocar erros: **limite atingido** e **livro indisponível**
5. Abrir um **segundo cliente sem login** — a função protegida é recusada
6. **Derrubar o servidor** — novas tentativas e aviso no cliente



CONCLUSÃO

O que o projeto entrega e até onde ele vai

Entregue

✓ As 5 funções do enunciado, com as mensagens originais

✓ Contrato em IDL e stubs gerados (gRPC + Protocol Buffers)

✓ Concorrência, segurança e tolerância a falhas

✓ 63 testes de ponta a ponta, todos passando

Limitações

O servidor é um **ponto único de falha**: sem réplicas, não há para onde redirecionar.

Próximo passo: replicação (primário-backup) e *streaming* para avisar os clientes.

Obrigado! Perguntas?

`github.com/NathanMarts/biblioteca-rpc`